PROJECT AND TEAM INFORMATION

Project Title

RESQNET: Smart Urban Emergency & Transportation Network Simulator

Team Name: *Team #*	*RESQNET* *DSCPP-III-2026-T070*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Chhavi – 2510012506* *2510012506@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Tapashya – 2510011649* *2510011649@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Chahal, Arpit – 2510012000* *2510012000@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

RESQNET is a C++17 console-based simulator that models how a city responds to several emergencies at the same time. The city is represented as a weighted graph in which junctions, hospitals and key locations are nodes and roads are edges carrying distance and travel-time weights. The simulator prioritizes incoming incidents with a priority queue, selects suitable emergency vehicles from a shared resource pool, and routes them to the incident using Dijkstra's algorithm.

The overall goal is one integrated model that shows the connected decisions of emergency response: incident prioritization, resource allocation, traffic-aware routing, dispatch, rerouting around road failures, hospital selection and critical-junction analysis. The project focuses on algorithmic decision-making using Data Structures and OOP concepts, not on live GPS or hardware.

Updated Project Approach and Architecture (2 pts)

The system follows the layered architecture from our proposal: an input layer, a core system layer and a data layer, with a user interface (UI) layer for simulation controls and output display. The core flow is Disaster Event → Incident Analysis → Priority Engine → Resource Allocation → Routing Engine → Dispatch & Response → Hospital Selection → Network Analysis → Logging.

Currently implemented: the weighted city graph, the Event Manager and incident handling, the Priority Engine (priority queue), the Resource Allocation engine, the Routing Engine (Dijkstra) and the Dispatch & Response step.

Data structures and algorithms: weighted graph, priority queue, vectors, queues and hash maps; Dijkstra for routing, with BFS/DFS and articulation points planned for network analysis. OOP is used for Vehicle, Disaster and Hospital classes with inheritance, polymorphism and encapsulation.

Libraries and tools: C++17 with the Standard Template Library only (no external libraries); VS Code / Code::Blocks as IDEs; Git and GitHub for version control. The application runs as a single process, so modules communicate through function calls and shared in-memory data and no network protocol is used.

Tasks Completed (7 pts)

Task Completed	Team Member
Problem statement finalized; disaster and vehicle types identified; system architecture and class structure designed	All members (Chhavi as Team Lead)
City road network implemented in C++ as a weighted graph (junctions, roads, distance and travel-time weights)	Chhavi
Dijkstra's shortest-path routing implemented	Chhavi
Disaster/incident handling and emergency priority queue implemented	Chahal, Arpit
Resource allocation (selecting a suitable available vehicle for an incident) and OOP classes	Tapashya
Dispatch of selected resources to the incident	Tapashya
User interface (UI) for running and viewing the simulation	Chhavi

Challenges/Roadblocks (7 pts)

- Keeping routes correct when edge weights change. Travel cost must react to traffic and road failures, so the plan is to keep all weights inside one graph object and recompute Dijkstra whenever a weight changes.
- Integrating modules written by different members (priority queue, allocation, routing, dispatch). We agreed on common class interfaces and merge work in small steps using Git branches.
- Several incidents competing for the same vehicles. Allocation uses incident priority together with vehicle availability and distance, and will be tested with many simultaneous incidents.

Tasks Pending (7 pts)

Task Pending	Team Member (to complete the task)
Hospital selection by capability, capacity and travel time	Tapashya
Dynamic rerouting on road or junction failure	Chhavi
Critical junction detection using articulation points	Chhavi
Traffic congestion model and time-based simulation clock	Chahal, Arpit
File logging (fstream) of events and results	Chahal, Arpit
Scenario testing, comparison of results, documentation and final demo	All members

Project Outcome/Deliverables (2 pts)

- A complete C++17 console simulator with modules for emergency response management, smart traffic management and dynamic navigation / network analysis.
- The city modeled as a weighted graph of roads, junctions, hospitals and emergency points, with a multi-vehicle resource pool.
- Source code, project report/documentation with system design and flow diagrams, test results and the final project presentation.

Progress Overview (2 pts)

Of the 13 milestones in our proposal, milestones 1 (simulation rules), 2 (base city map) and 4 (emergency resources and allocation) are complete, and milestone 3 (incident lifecycle) is in progress: creation, prioritization and dispatch work, while arrival and closing remain. The core pipeline Disaster → Priority Queue → Resource Allocation → Dijkstra Routing → Dispatch is working.

The Phase II targets from our roadmap (graph, priority queue, Dijkstra routing, resource allocation) are met, and dispatch and the UI were also completed. Milestones 5 to 13 (traffic and congestion, intersection priority, rerouting, hospital selection, network analysis, full simulation, scenario comparison and final testing) are pending and planned for the next phase.

Codebase Information (2 pts)

Repository link: https://github.com/mchhavi929-ship-it/Resqnet Branch: main

Testing and Validation Status (2 pts)

Test Type	**Status (Pass/Fail)**	**Notes**
Graph construction	Pass	City graph built with junctions and roads; node count, edges and distance/travel-time weights checked against the input data.
Dijkstra shortest route	Pass	Routes between junction pairs compared with manually calculated shortest paths on a small sample graph.
Priority queue ordering	Pass	Incidents of different severity inserted in random order; the highest-priority incident is always removed first.
Multiple incident handling	Pass	Several simultaneous incidents of different disaster types created and processed in priority order.
Resource allocation	Pass	Selected vehicle matches the disaster type, is available and is the best choice by distance.
Dispatch	Pass	Dispatched vehicle is marked busy and is not assigned to another incident until it is free.
Integrated flow (end to end)	Pass	Disaster → priority queue → allocation → routing → dispatch run with sample city data.
Dynamic rerouting, hospital selection	Pending	Not tested yet; these features are still to be implemented.

Deliverables Progress (2 pts)

Deliverable	**Status**
C++ simulator: graph, priority queue, Dijkstra routing, resource allocation, dispatch, UI	Completed
C++ simulator: hospital selection, rerouting, traffic model, articulation points, simulation clock, logging	Pending
System design and flow diagrams	Completed (architecture diagram); flow diagrams in progress
Project report / documentation	In progress
Test results and scenario comparison	Pending
Project presentation	Phase I completed; final presentation pending